

deskly

Rapport final complet et factuel

Audit produit, architecture Flutter, Firebase, synchronisation pro-client, qualite et
feuille de route.

Application	Deskly / ClientFlow Pro
Repository analyse	/Users/david/dev/flutter_projects/clientflow_pro
Date	10 septembre 2026
Mode	Lecture seule - aucune modification fonctionnelle
Validation	flutter analyze: OK - flutter test: 33 tests passes

1. Resume produit

Deskly est une application Flutter de gestion client/projet destinee aux professionnels, petites entreprises, artisans, agences et independants. Elle centralise clients, projets, taches, actions prioritaires, portail client, messagerie, documents, demandes de documents, validations, timeline, PDF, notifications internes et themes clair/sombre.

Le produit va au-dela d'un CRM simple : il construit un espace de collaboration entre professionnel et client autour de l'avancement, des documents, des messages et des elements a valider.

2. Parcours global

- Demarrage par SplashPage, puis orientation selon Firebase Auth et role detecte.
- Parcours public : onboarding, choix de role, connexion pro, inscription pro, connexion client, invitation client, suivi par code.
- Parcours professionnel : dashboard, clients, projets, taches, messages, reglages.
- Parcours client : accueil, avancement, messages, documents, profil, detail projet et validations.
- GoRouter applique une thematisation separee : AppRoleTheme.professional et AppRoleTheme.client.

Routes principales

Zone	Routes
Public	/, /onboarding, /role-choice, /login, /register, /track, /client/login, /client/invite
Professionnel	/main, /clients/add, /clients/:clientId, /projects/add, /projects/:projectId, /tasks/add, /messages, /settings
Client	/client/home, /client/progress, /client/messages, /client/documents, /client/profile, /client/projects/:projectId

3. Authentification et roles

Firebase Authentication est utilise pour les comptes professionnels et clients. Le role est determine par `users/{uid}.role` lorsque disponible, puis par la presence d'un document `client_accounts/{uid}`.

Element	Constat
Compte professionnel	Cree dans <code>users/{uid}</code> avec role professional, nom, email et createdAt.
Compte client	Cree depuis un code d'invitation, puis lie a <code>client_accounts/{clientId}</code> .
Invitation	<code>client_invitations/{token}</code> et compatibilite legacy avec <code>shared_clients/{token}</code> .
Session	GoRouterRefreshStream ecoute <code>authStateChanges</code> et redirige selon le role.
Champs client clefs	<code>uid</code> , <code>clientId</code> , <code>professionalId</code> , <code>email</code> , <code>displayName</code> , <code>role</code> , <code>projectIds</code> , <code>sharedClientId</code> .

Fichiers principaux : `lib/data/services/auth_service.dart`, `lib/data/services/client_portal_service.dart`, `lib/data/models/client_account_model.dart`.

4. Espace professionnel

Dashboard

- Affiche l'accueil professionnel, les KPI, l'etat des projets, les actions prioritaires, Morning Brief et activite client si disponible.
- Project Pulse est presente en UI comme etat des projets : en bonne voie, en attente client, a surveiller.
- Les donnees viennent des providers clients, projets, taches, actions, documents et pulse.

Clients

- Liste avec recherche, filtres, FAB compact, compteur clients/actifs.
- Fiche client structuree en Vue d'ensemble, Projets, Documents, Portail.
- Notes independantes avec creation, modification et suppression.
- Portail client affiche directement statut, explication du code, code, copie code/lien et revocation.

Projets

- Liste projets avec recherche, filtres de statut, detail projet et export PDF.
- Creation manuelle ou via templates.
- Detail projet : Synthese, Travail, Validations.
- Validations remplace Livrables dans l'interface, mais les modeles techniques restent Deliverable.

Taches, messages, documents, reglages

- Taches pro dans users/{professionalUid}/tasks, avec synchronisation portail client.
- Messages pro/client via shared_clients/{sharedClientId}/messages.
- Documents : demandes de documents, upload, validation/refus, suppression selon l'interface actuelle.
- Reglages : profil, theme, notifications, legal, deconnexion.

5. Espace client

Page	Role actuel
Accueil	Projet principal, progression, etape actuelle/prochaine, actions a faire, documents a fournir.
Avancement	Detail du projet cote client avec synthese, travail, validations et timeline visible.
Messages	Conversation avec le professionnel via le meme fil shared_clients que le pro.
Documents	Demandes de documents et envoi de fichiers par le client.
Validations	Elements a valider, versions, ouverture fichier/lien, validation ou demande de modification.
Profil	Compte client, projets accessibles, deconnexion.
Notifications	Notifications in-app Firestore filtrees par recipientUid.

Le portail client possède une identité visuelle dédiée, plus calme, avec ClientTheme et AppRoleTheme.client.

6. Synchronisation pro-client

Donnée	Source principale	Lecture client	Etat
Projets	users/{proUid}/projects	Même projet via clientId/projectIds/sharedClientId	OK
Tâches	users/{proUid}/tasks	Filtres par internal/private	OK
Actions	users/{proUid}/project_actions	visibleToClient et assignedTo client	OK
Messages	shared_clients/{sharedClientId}/messages	Même thread pro/client	OK
Documents	Storage partagé + client_accounts/{clientId}/documents	Architecture hybride	PARTIEL
Validations	users/{proUid}/deliverables et deliverable_versions	Même source lue par client	PARTIEL
Timeline	users/{proUid}/timeline + copie client_accounts	Copie visible client	PARTIEL
Notifications	notifications	recipientUid	OK

7. Firebase et backend

Configuration

- Projet Firebase configure : app-clientflow-pro.
- Bucket configure : app-clientflow-pro.firebasestorage.app.
- firebase_options.dart et GoogleService-Info.plist iOS correspondent au même projet.
- firebase.json pointe vers firestore.rules et storage.rules.

Collections observées

Collection	Usage
users/{uid}	Profil utilisateur et rôle.
users/{uid}/clients	Clients du professionnel.
users/{uid}/projects	Projets source côté professionnel.
users/{uid}/tasks	Tâches source côté professionnel.
users/{uid}/project_actions	Action Center.

Collection	Usage
users/{uid}/document_requests	Demandes de documents source pro.
users/{uid}/deliverables	Validations/livrables.
users/{uid}/deliverable_versions	Versions de validations.
users/{uid}/deliverable_annotations	Annotations visuelles.
client_accounts/{clientId}	Compte portail client.
shared_clients/{sharedClientId}/messages	Messagerie commune.
shared_projects/{shareId}	Snapshots publics de suivi par code.
notifications	Notifications internes.

Risque P0 : storage.rules est encore en regle temporaire ultra-ciblee sur shared_clients/9AVELB avec deux UID codes en dur. Cette regle est utile au diagnostic mais ne doit pas partir en production.

8. Architecture Flutter

- Architecture par features : auth, dashboard, clients, projects, tasks, messages, client_portal, documents, deliverables, notifications, timeline, settings.
- Riverpod structure les repositories, services, controllers et providers d'ecran.
- GoRouter gere les routes et redirections role-based.
- FirestoreService centralise les collections utilisateur professionnelles via users/{uid}/{collection}.
- Les modeles sont simples, serialisables, avec getters utiles et copyWith sur les donnees evolutives.

9. Technologies et packages

Package	Usage
flutter_riverpod	State management et providers.
go_router	Navigation et redirections.
firebase_core	Initialisation Firebase.
firebase_auth	Authentification pro/client.
cloud_firestore	Base de donnees temps reel.
firebase_storage	Fichiers documents/validations.
file_picker	Selection de fichiers.
url_launcher	Ouverture liens/fichiers externes.
pdf + printing	Generation et partage PDF projet.

Package	Usage
shared_preferences / flutter_secure_storage	Preferences et donnees locales sensibles.

10. UI/UX et design system

Les couleurs sont centralisees dans AppTheme et AppRoleTheme. Le cote pro garde l'identite Deskly mint/teal. Le cote client utilise un background plus calme, des surfaces douces et reserve le vert principal aux CTA, progressions et etats actifs.

Zone	Palette dominante
Professionnel	#31BB80, #203E40, #F6F8F7, #FFFFFF, #DDE7E3
Client	#F4F8F6, #203E40, #70827D, #2B5954, #6FAE96, #31BB80
Composants	Cards, search field, filter tabs, badges, notification bell, empty states, header buttons.

11. Progression et statut projet

La source technique actuelle est ProjectTimelineProgressService. Elle calcule project.progress et project.status a partir de timeline events, actions et taches. Le statut derive de la progression : 0% = Planifie, entre 0% et 100% = En cours, 100% = Termine.

Ecart produit a noter : une demande precedente visait une progression basee uniquement sur les etapes. Le code actuel combine etapes, actions et taches. Il faut trancher cette regle avant production.

12. Project Templates

Les templates sont configures localement dans lib/data/templates/project_templates.dart. Ils peuvent creer un projet, une timeline, des taches, des actions client et des demandes de documents dans un batch Firestore.

Template	Contenu typique
Site internet	Brief, contenus, maquette, validation, developpement, recette, livraison.
Identite visuelle	Brief, moodboard, pistes creatives, validation, livraison des fichiers.
Application mobile	Cadrage, UX/UI, developpement, tests, publication.
Shooting photo	Existe dans le code mais peut ne pas etre affiche dans la selection actuelle.
Projet architecture	Existe dans le code mais peut ne pas etre affiche dans la selection actuelle.
Campagne marketing	Existe dans le code mais peut ne pas etre affiche dans la selection actuelle.
Personnalise	Garde le comportement manuel sans template.

13. Tests et qualite

Commandes executees pendant l'audit :

Commande	Resultat
flutter analyze	No issues found - execution OK.
flutter test	33 tests passes - All tests passed.

Tests presents

- client_note_model_test.dart
- client_portal_models_test.dart
- deliverable_annotation_model_test.dart
- deliverable_models_test.dart
- document_request_model_test.dart
- morning_brief_model_test.dart
- project_pulse_service_test.dart
- project_templates_test.dart
- project_timeline_progress_service_test.dart
- project_waiting_status_service_test.dart
- timeline_event_model_test.dart
- widget_test.dart

Limites de couverture

- Peu ou pas de tests d'integration Firebase reels.
- Pas de parcours E2E complet professionnel vers client.
- Regles Firestore/Storage non testees automatiquement.
- Peu de tests UI complexes sur les formulaires et modales.

14. Etat fonctionnel aujourd'hui

Fonction	Etat	Commentaire
Auth pro	OK	Creation et connexion presentes.
Auth client invitation	OK	Code invitation et compte client presentes.
Clients	OK	Liste, fiche, notes, portail.
Projets	OK	Creation, liste, detail, synchronisation.
Taches pro/client	OK	Client lit les taches non internes/privées.
Messagerie	OK	Source commune shared_clients/messages.

Fonction	Etat	Commentaire
Actions client	PARTIEL	Fonctionnelle mais dependante de regles complexes.
Documents	PARTIEL	Upload Storage OK en diagnostic, Firestore hybride.
Validations	PARTIEL	Flux present mais historique de bugs permission/lifecycle.
Timeline	PARTIEL	Donnees visibles, mais copie client encore utilisee.
PDF projet	OK	Service existant et branche UI detectee.
Notifications internes	OK	Firestore, badge non lu.
Regles Storage production	RISQUE	Regle temporaire codee en dur.

15. Bugs et limites actuelles

- storage.rules est volontairement temporaire et limite a shared_clients/9AVELB avec deux UID fixes.
- Certaines donnees client sont encore miroir/copie dans client_accounts, ce qui peut creer des decalages.
- La progression actuelle combine timeline, actions et taches au lieu d'etre strictement basee sur les etapes.
- Des logs temporaires restent visibles dans les providers/services client.
- Les regles Firestore sont puissantes mais difficiles a maintenir sans tests de securite automatises.
- shared_projects et shared_clients ont des lectures publiques par code, a valider selon le niveau de confidentialite attendu.
- Les validations ont connu des problemes de permissions et de lifecycle Flutter ; le flux existe mais doit etre teste E2E.

16. Differentiateurs produit

- Portail client separe de l'espace professionnel.
- Actions client/professionnel pour comprendre qui bloque le projet.
- Demandes de documents structurees.
- Validations de livrables avec versions.
- Timeline projet visible cote client.
- Project Pulse et Morning Brief.
- Messagerie integree au contexte client/projet.
- Rapport PDF projet.

17. Valeur commerciale

Deskly repond a un probleme clair : reduire les allers-retours disperses entre email, WhatsApp, Drive et messages informels. L'app donne au professionnel un centre de pilotage et au client un espace clair pour suivre, fournir, valider et communiquer.

Cible	Valeur
Freelances	Centraliser clients, projets, validations et relances.
Agences	Structurer la relation client et les documents.
Artisans / petites entreprises	Offrir un suivi professionnel sans outil complexe.
Architectes / designers	Partager et valider des etapes, documents et livrables.

18. Competences techniques demontrees

- Flutter multi-ecrans avec architecture par feature.
- Riverpod, AsyncNotifier, FutureProvider et StreamProvider.
- GoRouter avec redirection selon role.
- Firebase Auth, Firestore, Storage et rules.
- Synchronisation temps reel pro-client.
- Upload fichiers, generation PDF, notifications internes.
- Modeles, repositories, services et tests unitaires.

19. Roadmap recommandee

Priorite	Actions recommandees
P0	Remplacer storage.rules temporaire par regles dynamiques securisees. Nettoyer logs temporaires. Verifier validations/documents en E2E. Ajouter tests de rules Firebase.
P1	Choisir la source exacte de progression. Simplifier les miroirs client_accounts. Ajouter tests integration pro-client. Stabiliser ouverture fichiers PDF/images.
P2	Notifications push FCM. Back-office templates. Annotations PDF. Analytics produit. Activity feed complet.

20. Plan pour un futur PDF commercial

- Couverture Deskly avec proposition de valeur.
- Probleme marche et cible.
- Solution et parcours pro/client.
- Fonctionnalités clefs illustrees par captures.
- Architecture technique simplifiée.
- Sécurité Firebase et qualité.
- Roadmap et conclusion commerciale.

Conclusion : Deskly possède déjà une base produit ambitieuse et différenciante. Avant une démonstration fiable, la priorité est de stabiliser Storage, réduire les duplications de données et valider le parcours complet pro-client.